트램 위험 알림 로드맵 v2 (예산 60만원 · 라즈베리파이 5 + 노트북)

2026-10-07 · 동아리 트램 캡스톤 · 원본 문서: claude.ai/code/artifact/ba477f72-5184-4647-aecb-31bdb74bfb12

라즈베리파이 5는 차량(모형) 위에서 카메라·판단·경보를 맡고, 무거운 인식(YOLO 검출·선로 검출)은 노트북이 무선으로 받아 처리합니다. 예산은 약 48.5만원을 쓰고 약 11.5만원을 예비비로 남기며, 큰 틀(4단계 경보, 10주 일정, 1:10 모형 시험)은 기존 로드맵 그대로입니다.

목차

1. 파이+노트북 분담, 괜찮은가
2. 장비와 예산표
3. 연산 분담 구조
4. 사용할 알고리즘 (4-1. 코드 구조와 통신 형식)
5. 기존 로드맵에서 달라지는 것
6. 모델 경량화, 우리 실력으로 되나
7. 10주 일정 (7-1. 역할 나누기 · 7-2. 주차별 상세 계획 · 7-3. 시험 시나리오와 측정 방법)
8. 위험 요소와 대안
9. 이번 주에 정할 것

1. 파이+노트북 분담, 괜찮은가

좋은 선택입니다. 60만원 안에서 라즈베리파이 5 혼자 YOLO를 10 FPS 이상 돌리기는 빠듯한데, 무거운 인식을 노트북(Core Ultra 7 255H, Arc 140T)에 넘기면 성능 걱정이 거의 사라집니다. 보드에서 AI 가속이 필요 없어지니, 이전 안에서 추천한 RDK X5보다 자료가 훨씬 많은 라즈베리파이 5가 오히려 낫습니다.

다만 세 가지 조건을 지켜야 합니다.

- **무선 지연을 재고 관리하기:** 영상을 보내고 결과를 받는 데 시간이 듭니다. 전용 5 GHz 공유기를 쓰고, 2주차에 왕복 지연을 직접 재서 0.3초 목표 안에 드는지 확인합니다.
- **끊겨도 안전하게:** 노트북 결과가 0.3초 넘게 안 오면 파이가 화면과 경보를 "감시 불가"로 바꿉니다. 여유가 되면 파이가 작은 모델로 혼자 느리게라도 감시합니다.
- **나누는 선을 깔끔하게:** 기존 로드맵의 층 구조 그대로 "인지층은 노트북, 판단층·출력층은 파이"로 나눕니다. 그래야 코드를 나눠 만들기 쉽고, 발표에서도 "판단은 차량 위 보드가 한다"고 설명할 수 있습니다.

2. 장비와 예산표

약 48.5만원을 쓰고 약 11.5만원을 예비비로 둡니다. 가격은 제가 알던 대략적인 값이고, 라즈베리파이는 메모리 가격 때문에 가격이 자주 바뀝니다. 발주 전에 국내 판매가를 꼭 확인해 주세요.

구분	품목	사양 · 이유	예산(원)
보드	라즈베리파이 5	8GB 권장. 연산을 나누므로 4GB도 충분(아끼면 예비비로)	150,000
보드	공식 27W 전원	USB-C 5V 5A, 실내 개발용	20,000

구분	품목	사양 · 이유	예산(원)
보드	액티브 쿨러 · 케이스	연속 동작 시 발열 대비	20,000
보드	microSD	64 GB, A2	15,000
카메라	라즈베리파이 카메라 모듈 3 + 파이 5용 케이블	표준형(화각 약 66°), HDR, CSI 연결이라 지연이 적고 USB 전류를 안 씀	50,000
통신	5 GHz 공유기	학교 와이파이 대신 우리 전용 망	30,000
통신	노트북용 USB 랜 어댑터 + 랜선	책상 시험용 유선 연결, 지연 비교	20,000
전원	보조배터리	USB-C PD, 5V 3A 이상. 주행체 위 파이용	40,000
경보	부저 · LED · 점퍼선	파이 GPIO에 바로 연결	10,000
모형	주행체	RC카 또는 라인트레이서 + 엔코더(속도 정답)	80,000
모형	시험장 재료	테이프(궤간 14.35 cm), 피규어, 거리 눈금, 체커보드	30,000
모형	거치대	카메라 · 파이 고정	20,000
여유	예비비	가격 변동, 고장. 분담이 안 되면 AI HAT+ 구입	115,000
	합계		**600,000**

사지 않는 것: OAK-D Lite(약 32만), 7인치 화면(노트북 화면 사용), 차량용 12V 전원 세트. 개발 초반에는 휴대폰으로 찍은 영상을 그대로 씁니다.

3. 연산 분담 구조

파이는 보고(촬영) 판단하고 알리며, 노트북은 무엇이 어디 있는지 인식합니다. 한 프레임이 돌아오는 데 약 0.1~0.2초로 예상하며, 목표인 0.3초 안에 들어야 합니다. 아래 시간은 추정이라 2주차에 직접 재서 고칩니다.

순서	어디서	하는 일	시간(추정)
1	파이	카메라 모듈 3으로 1280×720, 30 fps 촬영	약 35 ms
2	파이 → 노트북	JPEG로 압축(약 60~100 KB)해 프레임 번호·시각과 함께 전송 (ZeroMQ, 5 GHz 와이파이)	20~60 ms
3	노트북	YOLO11n(OpenVINO)로 사람·자전거·차 검출 + ByteTrack 추적	15~30 ms
4	노트북	선로 검출(모형은 OpenCV 테이프 검출)	10~20 ms
5	노트북 → 파이	박스·ID·레일 좌표를 JSON(1 KB 미만)으로 회신	5~20 ms
6	파이	거리·속도·TTC·위험 등급 계산	5 ms 이하
7	파이 / 노트북	파이는 부저·LED·로그, 노트북은 운전자 화면 표시	약 10 ms

- **배치:** 파이·카메라·보조배터리는 주행체 위에, 노트북은 시험장 옆 책상에 둡니다. 모형 시험장(약 8 m) 정도는 공유기 하나로 충분합니다.
- **오래된 결과는 버림:** 파이는 프레임 번호로 시간을 맞추고, 촬영한 지 0.3초가 넘은 결과는 쓰지 않습니다.
- **감시 불가 표시:** 0.3초 동안 결과가 없으면 회색 "감시 불가"와 별도 경고음을 냅니다. 기존 로드맵의 "안전 쪽으로 실패" 원칙입니다.
- **파이 단독 모드(선택):** 연결이 끊기면 파이가 경량화한 YOLO로 직접 검출합니다. 느리더라도 감시는 계속합니다(6장).

4. 사용할 알고리즘

알고리즘은 기존 로드맵과 같고, 깊이 카메라가 빠지면서 거리를 레일 폭(궤간)으로 재는 방식이 주 방법이 됩니다. 팀이 가진 예제 코드(`code_examples` 02~06번)가 이미 이 구조입니다.

단계	알고리즘	어디서	예제
① 선로 검출	모형: OpenCV 색 분리(HSV) → 행별 레일 점 → 다항식 맞춤, 몇 프레임 평활. 실제 영상(도전): RailSem19로 학습한 YOLO11n-seg	노트북	03
② 검출	YOLO11n 사전학습(COCO) 사람·자전거·차·오토바이·버스·트럭. 먼 곳은 선로 먼 쪽만 잘라 한 번 더 검출	노트북	02
③ 추적	ByteTrack (Ultralytics track): 프레임마다 같은 물체에 같은 ID	노트북	02
④ 거리	앞 거리 Z = f × 1.435 m ÷ (물체 발 높이에서의 레일 간격 px). 옆 거리 = (물체 x − 레일 중심 x) × 1.435 ÷ 레일 간격 px. 레일이 안 보이면 사람 키 1.7 m로 백업	파이	04
⑤ 운동	최근 10프레임 거리의 기울기로 접근 속도, TTC = 거리 ÷ 접근 속도, 필요 감속도 = v^2 ÷ 2d	파이	05
⑥ 위험 등급	통로 안/밖 × TTC(5초·3초) × 필요 감속도(1.2 m/s²) → 안전·주의·경고·위험. 올릴 때 2프레임, 내릴 때 1초 유지로 깜빡임 방지	파이	05
⑦ 출력	화면 색 띠·박스, 부저·LED, 경고 이상 이벤트 기록(events.csv)	파이 + 노트북	06

새로 만들 코드는 두 개라서 크지 않습니다. 파이의 `pi_client.py`(촬영·전송·판단·경보)와 노트북의 `laptop_server.py`(수신·인식·회신)이며, 안에서는 기존 02~05번 함수를 그대로 불러 씁니다.

4-1. 코드 구조와 통신 형식

저장소 하나에 파이용·노트북용 코드를 같이 두고, 두 쪽이 같이 쓰는 설정과 메시지 형식은 `common/`에 한 번만 씁니다. 괄호 안 번호는 지금 예제 코드에서 가져올 파일입니다.

```
tram-guard/
├─ config/      model.yaml (1:10 모형) · real.yaml (실제 크기)
├─ common/      config.py · protocol.py (메시지 형식) · timing.py
├─ laptop/      laptop_server.py (메인)
│  ├─ detector.py   YOLO 검출·추적 (02)
│  ├─ rails.py      선로 검출 (03)
│  └─ ui.py         운전자 화면 (06)
├─ pi/          pi_client.py (메인)
│  ├─ camera.py     picamera2 촬영
│  ├─ distance.py   거리 (04)
│  ├─ risk.py       TTC·등급·깜빡임 방지 (05)
│  ├─ alarm.py      부저·LED (gpiozero)
│  ├─ logger.py     events.csv 기록
│  └─ fallback.py   파이 단독 모드 (선택)
├─ tools/       latency_test.py · fps_bench.py · calibrate.py · eval_scenario.py
├─ data/        videos/ · logs/   (Git에 올리지 않음)
└─ docs/        회의록 · decisions.md · 실험 기록
```

파이 → 노트북 (영상): 메시지 하나에 짧은 머리말(JSON)과 JPEG를 붙여 보냅니다. 노트북은 밀린 프레임을 쌓아 두지 않고 가장 최근 것만 처리합니다(ZeroMQ PUSH/PULL + CONFLATE).

```
{"frame_id": 1532, "t_capture": 1791362400.123, "w": 1280, "h": 720}
```

노트북 → 파이 (인식 결과): 파이는 `t_capture`를 자기 시계로 다시 비교해 지연을 재고, 0.3초가 넘은 결과는 버립니다. 두 장치의 시계는 서로 다르므로 지연은 항상 파이 시계 하나로만 재야 합니다.

```
{"frame_id": 1532, "t_capture": 1791362400.123, "infer_ms": 18.4,
 "rails": {"ok": true, "left": [0.0012, -1.9, 1210], "right": [-0.0011, 1.7, 160], "y_top":
380, "y_bottom": 720},
 "objects": [{"id": 7, "cls": "person", "conf": 0.82, "box": [612, 401, 648, 498]}]}
```

파이 → 노트북 (판단 결과, 화면 표시용): 등급과 가장 위험한 물체의 숫자만 보냅니다.

```
{"frame_id": 1532, "level": 2, "rtt_ms": 142, "top": {"id": 7, "z_m": 18.2, "lat_m": 0.9,
"ttc_s": 3.4, "a_req": 1.1}}
```

5. 기존 로드맵에서 달라지는 것

큰 틀은 거의 그대로입니다. 바뀌는 것은 카메라와 계산하는 장소이고, 그 때문에 통신 모듈 하나가 늘고 가까운 거리의 깊이 측정이 빠집니다.

항목	기존 로드맵	바뀐 계획	영향
예산	100만원	60만원	깊이 카메라·화면·차량 전원 제외
카메라	OAK-D Lite (컬러 + 깊이, YOLO 내장)	카메라 모듈 3 (단안)	8 m 이내 깊이 측정이 없어짐 → 레일 폭 기반 거리가 주 방법
계산 장소	카메라 칩(검출) + 파이(나머지)	노트북(인지층) + 파이(판단·출력층)	통신 모듈 추가, 지연 측정이 새 체크포인트
운전자 화면	7인치 디스플레이	노트북 화면	화면 디자인은 그대로
전원	12V 배터리 + 5V 5A 컨버터	USB-C 보조배터리	카메라가 CSI라 USB 전류 제한 문제가 없음
그대로인 것	4층 구조, 4단계 경보, TTC 5초·3초, 성능 목표, 1:10 모형 시험장, 시나리오 S1~S8, 4·7·10주차 체크포인트, 부저·LED		

발표나 보고서에는 "실제 트램에서는 노트북 대신 차량용 연산 장치가 인지층을 맡는다"고 적으면 됩니다. 코드는 층으로 나뉘어 있어서 장치만 바꾸면 됩니다.

6. 모델 경량화, 우리 실력으로 되나

됩니다. 우리가 할 경량화는 모델을 새로 설계하는 게 아니라, 이미 있는 가장 작은 YOLO를 빠른 형식으로 바꾸고 입력을 줄이는 것이라 대부분 명령 한두 줄입니다. 게다가 무거운 계산은 노트북이 맡으니 경량화는 필수가 아니라, 파이 단독 모드와 발표용 비교 실험에 쓰는 더하기 점수입니다.

방법	하는 일	난이도	효과	우리 계획
가장 작은 모델	YOLO11n 사용	쉬움	기본	이미 사용 중
형식 변환	노트북은 `format=openvino`, 파이는 `format=ncnn`으로 export	쉬움 (명령 한 줄)	그냥 돌릴 때보다 눈에 띄게 빨라짐	2주차
입력 크기 줄이기	`imgsz` 640 → 416 또는 320	쉬움 (옵션 하나)	계산량이 면적에 비례해 320이면 약 1/4. 대신 먼 물체를 놓침	5주차, 파이 단독 모드에만

방법	하는 일	난이도	효과	우리 계획
INT8 양자화	export에 `int8=True` + 우리 영상 프레임 수백 장으로 보정	보통	더 빨라지고 정확도는 조금 떨어짐. 전후 비교 필수	5~6주차
우리 데이터로 재학습	모형 피규어·테이프 선로 사진으로 파인튜닝 (Colab)	보통	속도보다 정확도가 오름	피규어 검출이 약하면 6주차
가지치기 · 지식 증류 · 구조 설계	모델 내부를 고침	어려움	크지만 시간이 많이 듦	이번 범위 아님 (향후 과제)

파이 5 CPU 혼자서는 YOLO11n(NCNN, 640)이 초당 수 장에서 10장 안팎으로 알려져 있습니다. 제 기억에 의존한 값이라 5주차에 직접 재야 합니다. "PyTorch vs OpenVINO vs INT8"의 FPS·정확도 비교표는 그 자체로 좋은 발표 자료가 됩니다.

7. 10주 일정

체크포인트는 그대로(4주차 PC 데모, 7주차 MVP, 10주차 최종)이고, 2주차에 "분담 지연 측정" 관문을 하나 더했습니다. 1주차는 10월 5일, 3주차는 중간고사 주간으로 두었습니다.

주차	기간	알고리즘 (노트북)	보드·통신·모형 (파이)	끝났을 때 보여줄 것
1	10/5–10/9	가상환경 설치, 예제 00·06번 실행, 휴대폰 트램·모형 영상 촬영	가격 확인, 파이 5·카메라·공유기 발주	예산 확정, 발주 완료
2	10/12–10/16	YOLO OpenVINO 변환, 노트북 FPS 측정	파이 OS·카메라 확인, 파이→노트북 영상 전송 뼈대(ZeroMQ)	**관문: 왕복 지연 0.3초 이내인지**
3	10/19–10/23	(중간고사) 테이프 선로 검출 다듬기	부저·LED 연결 시험	—
4	10/26–10/30	거리·TTC·등급을 한 화면에	시험장 제작(테이프·눈금)	**PC 데모**: 녹화 영상에 통로·박스·거리·등급
5	11/2–11/6	인지층을 `laptop_server.py`로, 경량화 실험(입력 크기·INT8)	판단·출력층을 `pi_client.py`로	분담 구조로 실시간 동작, FPS·지연 표
6	11/9–11/13	끊김 감지·감시 불가 표시, (여유 시) 파이 단독 모드	주행체에 파이·카메라·배터리 탑재, 엔코더 기록	—
7	11/16–11/20	통합 디버깅	모형 주행 시험	**MVP**: 모형에서 실시간 경보
8	11/23–11/27	시나리오 S1~S8 시험	반복 주행·촬영	시험 데이터
9	11/30–12/4	오차 분석, 임계값 조정, 비교 실험(분담 vs 파이 단독)	유선 vs 무선 지연 비교	결과표·그래프
10	12/7–12/11	보고서·발표자료	시연 준비, 시연 영상	**최종 발표**

7-1. 역할 나누기

코드가 층으로 나뉘는 선을 그대로 역할의 선으로 씁니다. 두 쪽은 4-1의 메시지 형식만 맞추면 서로 기다리지 않고 동시에 만들 수 있습니다. 사람이 적으면 A와 담당 없는 공통 업무를 한 사람이 겸해도 됩니다.

역할	맡는 것	담당 파일	담당 체크포인트
A. 인지 (노트북)	YOLO 검출·추적, OpenVINO 변환, 선로 검출, 운전자 화면, 경량화 실험	`laptop/` 전체, `tools/fps_bench.py`	4주 PC 데모
B. 판단·보드 (파이)	파이 OS·카메라, 통신, 거리·TTC·등급, 부저·LED, 감시 불가, 로그	`pi/` 전체, `common/protocol.py`, `tools/latency_test.py`	2주 지연 관문
C. 모형·시험	주행체·엔코더, 테이프 선로·눈금, 피규어 움직이기, 측면 카메라, 시나리오 진행·정답 기록	`tools/eval_scenario.py`, `data/`	7주 MVP 시험
공통 (한 명 책임)	예산·발주, 회의록, decisions.md, 보고서·발표 취합	`docs/`	10주 최종

매주 운영: 주 1회 30분 회의에서 ① 지난주 완료 기준을 화면으로 확인하고 ② 막힌 것 하나씩 말하고 ③ 다음 주 할 일을 아래 7-2 표에서 고릅니다. 정한 숫자(임계값, 부품)는 그날 `docs/decisions.md`에 한 줄씩 남깁니다.

7-2. 주차별 상세 계획

주마다 목표 한 줄, 역할별 할 일(체크박스), 끝났다고 말할 수 있는 완료 기준을 적었습니다. 완료 기준을 못 맞추면 다음 주 첫 할 일로 넘기고, 두 주 연속 밀리면 8장의 대안으로 바꿉니다.

1주차 (10/5–10/9, 10/9 한글날) — 구성 확정과 발주

- ☐ 공통: 이 문서로 파이+노트북 분담 구조 확정, 역할 A·B·C 배정
- ☐ 공통: 국내 판매가 확인 후 파이 5 세트·카메라 모듈 3·파이 5용 케이블·공유기 발주
- ☐ 공통: GitHub 저장소와 4-1 폴더 구조 만들기, 회의 요일 고정
- ☐ A: 노트북에 가상환경 설치, 예제 00·06번 실행
- ☐ A: 휴대폰으로 공개 트램 영상 2~3개 확보, 테이프 선로 시험 영상 촬영(1x 카메라, 손떨림 보정 끔, 거치대 고정)
- ☐ B: 라즈베리파이 OS 설치·SSH 입문 자료 읽기, ZeroMQ 예제 노트북 두 창에서 돌려 보기
- ☐ C: 시험장 장소(약 8 m × 2 m)와 주행체(RC카/라인트레이서) 정하기

완료 기준: 발주 완료, 저장소에 첫 커밋, 팀원 전원 노트북에서 `06_pipeline.py`가 도는 것.

2주차 (10/12–10/16) — 관문: 분담이 제시간 안에 도는가

- ☐ A: `yolo export model=yolo11n.pt format=openvino`, `fps_bench.py`로 PyTorch vs OpenVINO FPS 비교 (640·416·320)
- ☐ A: 휴대폰 영상으로 YOLO 검출·추적, 사람 ID가 유지되는지 확인
- ☐ B: 파이 도착 시 OS(64비트)·SSH·카메라(`rpicam-hello`) 확인, 자동초점은 고정값으로 잠금
- ☐ B: `latency_test.py`: 파이가 720p JPEG를 보내고 노트북이 빈 결과를 바로 돌려주는 왕복 시간을 1,000회 측정
- ☐ B: 같은 시험을 공유기 5 GHz / 유선 랜으로 반복해 비교
- ☐ C: 체커보드 인쇄, 카메라 모듈 3의 실제 화각 측정(`calibrate.py`) 후 `HFOV_DEG` 갱신

완료 기준: 왕복 지연의 중앙값과 95% 값이 표로 나오고, 노트북 추론까지 더해 0.3초 안에 드는지 판단. 안 되면 이 주에 바로 전송 해상도를 줄이거나 Plan B(AI HAT+)를 정합니다.

3주차 (10/19–10/23, 중간고사) — 가볍게 유지

- ☐ A: 테이프 선로 검출(03번) 모형 영상에 맞게 색 범위·평활 조정
- ☐ B: 부저·LED를 GPIO에 연결, 등급 0~3별 소리·빛 패턴 만들기

☐ C: 시험장 재료 구입, 테이프 궤간 14.35 cm 시험 붙이기

완료 기준: 모형 영상 10개 중 9개 이상에서 선로가 잡히는 것, 부저가 등급별로 다르게 울리는 것.

4주차 (10/26–10/30) — PC 데모

☐ A: 녹화 영상 한 편에 선로·위험 통로·박스·ID·거리·TTC·등급 색 띠를 한 화면에 표시(06번을 모형 영상에 맞춤)
☐ A: 선로 먼 쪽만 잘라 한 번 더 검출하는 크롭 검출 추가, 먼 피규어 검출이 늘었는지 비교
☐ B: `common/protocol.py`에 4-1 메시지 형식 확정, 파이에서 카메라 → 노트북 실시간 전송(아직 인식 없이 화면만)
☐ C: 거리 눈금(50 cm 간격), 구역 테이프(6·10·20 cm), 측면 고정 카메라 설치로 시험장 완성
☐ C: 시험장에서 휴대폰으로 S1·S2·S3 영상 촬영(정답 거리 기록)

완료 기준 (체크포인트): 모형 영상에서 서 있는 피규어의 거리가 눈금 대비 ±10% 안, 등급이 안전→주의→경고→위험 순서로 오르는 데모 영상.

5주차 (11/2–11/6) — 분담 구조로 합치기

☐ A: 인지 코드를 `laptop_server.py`로 옮기기(수신 → 검출·추적·선로 → JSON 회신)
☐ A: 경량화 실험 1: INT8(`int8=True`, 모형 영상 프레임 300장으로 보정), FPS·검출 수 전후 비교
☐ B: 판단 코드를 `pi_client.py`로 옮기기(결과 수신 → 거리·TTC·등급 → 부저·LED → 판단 결과 송신)
☐ B: 프레임 번호 맞추기, 0.3초 넘은 결과 버리기, 프레임별 `rtt_ms` 로그
☐ C: 주행체에 엔코더 달고 일정 속도(약 1.4 m/s) 주행 확인

완료 기준: 책상 위에서 파이 카메라 앞에 피규어를 가까이 가져가면 부저가 울림. 10분 연속 동작 기준 10 FPS 이상, 지연 95% 값 0.3초 이하인 표.

6주차 (11/9–11/13) — 실제로 움직이며 돌리기

☐ B: 감시 불가: 0.3초 결과 없으면 회색 표시 + 별도 경고음, 공유기 전원을 뽑아 시험
☐ B: (여유 시) `fallback.py`: 끊기면 파이가 YOLO11n NCNN 320으로 직접 검출, 파이 단독 FPS 측정
☐ A: 등급 깜빡임 점검(올릴 때 2프레임, 내릴 때 1초), 경고 이상일 때 전후 영상 저장
☐ C: 파이·카메라·보조배터리를 주행체에 탑재(카메라 높이·각도 기록), 피규어를 줄이나 서보로 일정하게 가로지르게 만들기

완료 기준: 주행체가 달리는 상태에서 끊김 없이 1분 이상 동작, 공유기를 끄면 0.5초 안에 감시 불가 경보.

7주차 (11/16–11/20) — MVP

☐ 전원: S1(선로 위 정지 피규어)과 S3(옆에서 가로지름)을 각 5회 실시간 주행
☐ A·B: 끝난 뒤 로그에서 놓친 경보·잘못된 경보를 찾아 버그 목록 만들고 고치기
☐ C: 측면 카메라로 피규어가 통로에 들어간 시각 기록, 시연용 영상 1편 녹화
☐ 공통: 중간 점검 발표가 있으면 이 영상과 5주차 FPS·지연 표로 준비

완료 기준 (체크포인트): 모형에서 주행 중 S1·S3 10회 모두 위험 경보가 울림(놓침 0건).

8주차 (11/23–11/27) — 시나리오 시험

☐ C: 시나리오 S1~S8을 각 10회(조명 시험 S7은 밝음/어두움 각 5회), 주행 조건과 정답을 시험표에 기록
☐ A·B: 매 주행의 `events.csv`·영상·`rtt_ms` 로그를 `data/logs/날짜_시나리오_회차/`로 정리

☐ A: `eval_scenario.py`로 로그와 정답을 맞춰 지표 자동 계산

완료 기준: S1~S8 결과표 초안(7-3의 지표 모두 채움).

9주차 (11/30–12/4) — 분석과 비교 실험

☐ A: 거리 구간별 오차 그래프, TTC 5초·3초 기준 조정 시 오경보·놓침 변화
☐ A: 비교 실험: 통로 없음 vs 있음, TTC만 vs TTC + 옆 거리 + 필요 감속도
☐ B: 비교 실험: 분담(무선) vs 분담(유선) vs 파이 단독의 FPS·지연·놓침
☐ A: 경량화 비교표: PyTorch vs OpenVINO vs INT8 (FPS, 검출률)
☐ C: 부족한 시나리오 재시험

완료 기준: 발표에 넣을 그래프 3개 이상과 "왜 이렇게 설계했나"를 보여 주는 비교표 2개.

10주차 (12/7–12/11) — 최종 발표

☐ 공통: 보고서(문제 → 설계 → 구현 → 시험 결과 → 한계와 향후 과제), 발표 자료
☐ C: 시연 영상 2분 편집(화면 + 측면 영상 나란히)
☐ B: 시연 장소에서 리허설 2회, 예비 보조배터리·랜선 준비
☐ 공통: 향후 과제 정리(차량용 연산 장치, 매립형 레일 학습, 야간·악천후)

완료 기준: 라이브 시연이 실패해도 시연 영상으로 대체할 수 있는 상태.

7-3. 시험 시나리오와 측정 방법

시나리오는 기존 로드맵의 S1~S8을 그대로 쓰고, 분담 구조 때문에 S9(연결 끊김)를 추가했습니다. 각 10회가 기본입니다.

#	시나리오 (모형)	기대 결과	보는 지표
S1	선로 위 정지 피규어, 주행체 접근	주의 → 경고 → 위험 순서로 상승	리드타임, TTC 오차
S2	선로 옆(통로 밖)을 나란히 이동	경고 이상 없음	오경보율
S3	옆에서 선로로 가로지름 (−45° / 0° / 45°)	진입 전에 경고	놓침률, 리드타임
S4	들어왔다가 다시 나감	상승 후 해제, 깜빡임 없음	등급 변화 횟수
S5	여러 개 (피규어 2 + 차 1)	가장 위험한 것 기준 경보	ID 유지율
S6	곡선 구간	곡선 통로 정상 생성	선로 검출 성공률
S7	어두운 조명·역광	성능 저하 폭 측정	검출률 변화
S8	레일 일부 가림	사람 키 백업으로 전환	거리 오차
S9	주행 중 공유기 끄기	0.5초 안에 감시 불가 경보	감지까지 걸린 시간

지표	목표	재는 방법
처리 속도	10 FPS 이상	파이가 1초마다 받은 유효 결과 수를 로그에 기록, 10분 평균·최저
지연	95% 값 0.3초 이하	파이 시계로 `t_capture` → 결과 수신까지(`rtt_ms`), 중앙값과 95% 값
앞 거리 오차	±10% 이내	피규어를 눈금 위에 세우고 로그의 거리와 비교
옆 거리 오차	구역 테이프와 일치	6·10·20 cm 구역 테이프 위 위치와 비교
놓침	0건	S1·S3에서 측면 카메라상 진입 전 위험 경보가 없는 회차 수
오경보	10% 이하	S2에서 경고 이상이 나온 회차 비율

지표	목표	재는 방법
리드타임	클수록 좋음	경보 시각과 측면 카메라상 통로 진입 시각의 차이
연속 동작	2시간	보조배터리로 켜 두고 로그 끝 시각 확인

안전 원칙: 사람을 움직이는 차량 앞에 두는 시험은 하지 않고, 모두 피규어·입간판으로 합니다.

8. 위험 요소와 대안

위험	대안
와이파이 지연이 커서 0.3초를 넘음	전송 해상도·JPEG 품질을 낮춤. 안 되면 주행 시험만 긴 랜선 유선 연결
연결이 중간에 끊김	파이가 0.3초 감시 후 "감시 불가" 경보. 여유 시 파이 단독 모드
분담 구조 자체가 안 됨	예비비로 AI HAT+를 사서 파이가 직접 검출. 또는 카메라를 노트북에 바로 연결해 시험
카메라 하나라 먼 거리 오차가 큼	여러 프레임 평활, 선로 먼 쪽 크롭 검출, 체커보드로 화각 보정
가격이 예상보다 비쌈	파이 4GB로 낮추거나 예비비에서 충당, 주행체는 손으로 미는 수레로 대체
실제 도로의 매립형 레일 검출	모형은 테이프 선로로. 실제 영상은 향후 과제(RailSem19로 YOLO-seg 학습)

9. 이번 주에 정할 것

- ☐ 라즈베리파이 5 + 노트북 분담 구조로 확정할지
- ☐ 파이 8GB / 4GB, 카메라 모듈 3 / USB 웹캠 중 선택
- ☐ 국내 판매가 확인 후 발주 목록 확정
- ☐ 모형 주행체와 시험장 장소
- ☐ 역할: 노트북 인지(검출·선로), 파이 판단·통신·경보, 모형 시험장
- ☐ 팀원 모두 가상환경 설치하고 예제 실행